

3. 다양한 알고리즘

1) 유클리드 알고리즘

2) 재귀 알고리즘

3) 탐색 알고리즘

4) 정렬 알고리즘

3-1) 유클리드 알고리즘-1

- [정리 12-3] 최대공약수에 대한 정리
- 두 음이 아닌 정수 $a, b (a > b)$ 에 대해
 $r = a \bmod b$ 가 성립하면 다음이 성립한다.

$$\text{GCD}(a, b) = \text{GCD}(b, r)$$

(증명)

$$r = a \bmod b \text{ 이면 } a = bq + r \ (0 \leq r < b)$$

어떤 양의 정수 k 를 a, b 의 공약수라고 하면

$$a = km, b = kn, bq = knq \ (m, n \text{은 음이 아닌 정수})$$

$$r = a - bq = km - knq = k(m - nq) \rightarrow k \text{는 } a, b, r \text{의 약수}$$

(a, b 의 공약수 집합 = b, r 의 공약수 집합)

$$\therefore \text{GCD}(a, b) = \text{GCD}(b, r)$$

3-1) 유클리드 알고리즘-2

- [예제 12-7] 150과 165의 최대공약수?

$$\begin{aligned}\text{GCD}(165, 150) &= \text{GCD}(150, 15) \\ &= \text{GCD}(15, 0)\end{aligned}$$

∴ 150과 165의 최대공약수는 15

3-1) 유클리드 알고리즘-3

- 유클리드 알고리즘(Euclid Algorithm)

- 두 양의 정수 a, b 의 최대공약수를 찾는 알고리즘

```
1 algorithm gcd( $a, b$ ) {  
2     if  $a < b$  then  
3         tmp= $a$ ;  $a=b$ ;  $b=tmp$ ;  
4     endif  
5     while( $b \neq 0$ )  
6          $r=a \bmod b$ ;  $a=b$ ;  $b=r$ ;  
7     endwhile  
8     return  $a$ ;  
9 }
```

3-2) 재귀 알고리즘-1

- 재귀 알고리즘(Recursive Algorithm)
 - 문제를 해결하면서 어떤 항목이 이전에 사용되었던 자기 자신을 다시 호출해 사용하는 방법

$$\begin{cases} f(0) = 1 \\ f(n) = 1 + f(n-1), n \geq 1 \end{cases}$$

$$\begin{aligned} f(n) &= 1 + f(n-1) \\ &= 1 + \{1 + f(n-2)\} = 2 + f(n-2) \\ &= 2 + \{1 + f(n-3)\} = 3 + f(n-3) \\ &\quad \dots \dots = n + f(0) \\ &= n + 1 \end{aligned}$$

3-2) 재귀 알고리즘-2

▪ $f(n)=1+f(n-1)=n+1$

```
1 algorithm sum(int n){  
2   s=0;  
3   for i=0 to n  
4     s=s+1;  
5   next i  
6   return s;  
7 }
```

```
1 algorithm sum(int n){  
2   if n!=0 then  
3     return 1+sum(n-1);  
4   else  
5     return 1;  
6   endif  
7 }
```

-시간복잡도: $f(n)=n+1 \leq n+n=2n \quad \therefore O(n)$

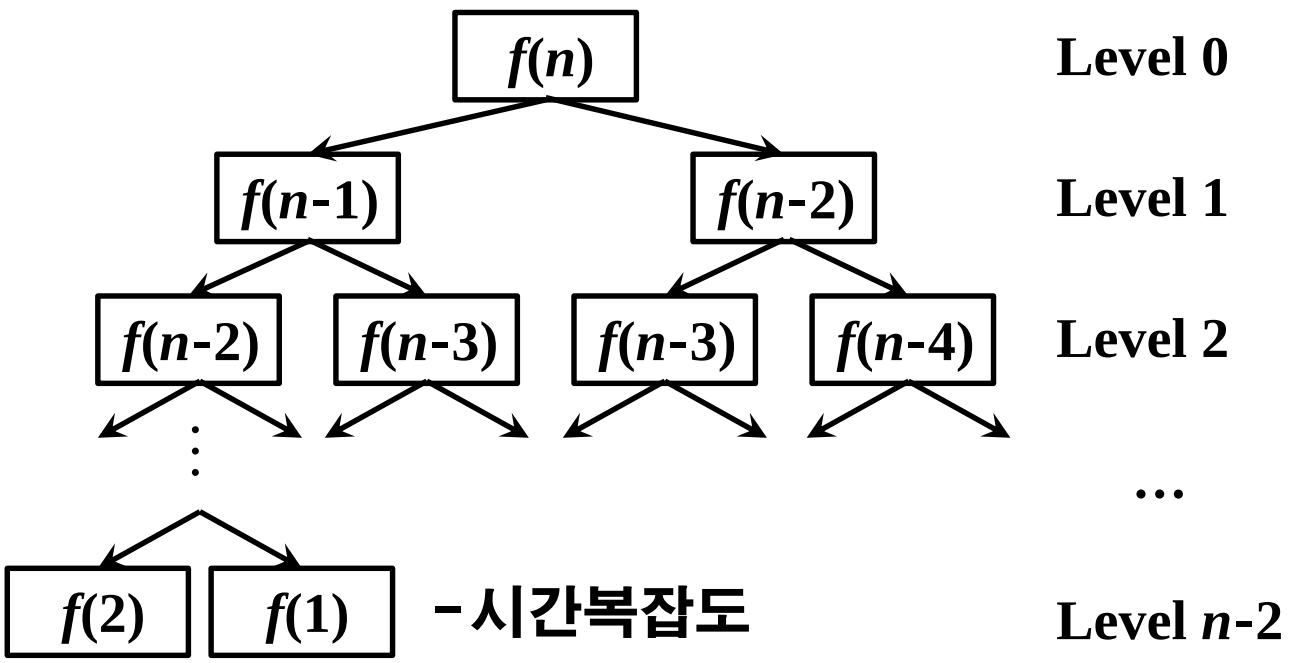
3-2) 재귀 알고리즘-3

- $f(1)=f(2)=1, f(n)=f(n-1)+f(n-2)$

```
1 algorithm fibonacci(int n){  
2     if (n == 1 || n == 2) then  
3         return 1;  
4     else  
5         return fibonacci(n-1) + fibonacci(n-2);  
6     endif  
7 }
```

3-2) 재귀 알고리즘-4

▪ $f(1)=f(2)=1, f(n)=f(n-1)+f(n-2)$



- 시간복잡도

$$f(n) \leq 2^{(n-3)+1} - 1 = 2^{n-2} - 1$$

$$\leq 2^{n-2} = 2^n / 4 \leq 2^n \quad \therefore O(2^n)$$